CS 450 HW 1 DEFINITIONS

Linear Algebra. Operations within vector spaces using vectors and matrices. Some linear algebra concepts that are required by the homework are listed here.

Cross Product: Computes a 3-element vector orthogonal to two other 3-element vectors. Written as

$$\mathbf{c} = \mathbf{a} \times \mathbf{b}.$$

Furthermore,

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\| \, \|\mathbf{b}\| \sin\theta,$$

where θ is the angle between $\mathbf{a}$ and $\mathbf{b}$. This is exactly the area of the parellelogram formed by $\mathbf{a}$ and $\mathbf{b}$. Note $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$. GLM provides the function `glm::cross` to compute the cross product.

Dot Product: Sum of the per-element product of two vectors. For vectors $\mathbf{a}$ and $\mathbf{b}$,

$$\mathbf{a} \cdot \mathbf{b} = \sum_i \mathbf{a}_i \mathbf{b}_i.$$

Can be used to compute the angle between two vectors. Formally, the angle θ between $\mathbf{a}$ and $\mathbf{b}$ is

$$\|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta = \mathbf{a} \cdot \mathbf{b}.$$

Note for vectors pointing in the same direction, the dot product is positive, for vectors pointing in the opposite direction, the dot product is negative, and for orthogonal vectors, the dot product is zero. GLM provides `glm::dot`.

Homogeneous Coordinates: Coordinate system allowing affine transformations to be represented by a matrix. When working in $\mathbb{R}^3$, achieved by lifting into $\mathbb{R}^4$. A 3D point $\begin{pmatrix} x & y & z \end{pmatrix}^T$ is stored as $\begin{pmatrix} x & y & z & 1 \end{pmatrix}^T$ in homogeneous coordinates and transformations as 4×4 matrices.

Matrix Inverse: The inverse of a matrix $\mathbf{A}^{-1}$ is the matrix such that

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$$

holds. Only square matrices are invertible and not all square matrices are invertible. Note $\mathbf{I} = \mathbf{I}^{-1}$. Furthermore,

$$(\mathbf{A}\mathbf{B})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}.$$

GLM provides `glm::inverse` to do invert a matrix.

Norm: Size of a vector, and also called the length or magnitude. Typically, we compute the L2 norm, computed as

$$\|\mathbf{v}\| = \sqrt{\mathbf{v} \cdot \mathbf{v}} = \sqrt{\sum_i \mathbf{v}_i^2}.$$

GLM provides the function `glm::length` to compute the norm.

Normalization: Process of resizing a vector such that it points in the same direction, but its norm is 1. For a vector $\mathbf{v}$, its normalized version is

$$\mathbf{v}_{\text{normalized}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}.$$

Normalized vectors are commonly used to encode purely directional information. Vectors with norm 0 cannot be normalized. GLM provides a function to perform this through the function `glm::normalize`.

Transformation Matrix: Matrix transforming between different spaces. Typically, we work in homogeneous coordinates, placing our vectors in $\mathbb{R}^4$. More relevant when discussing common spaces in graphics, explained later. Transformation matrices can be combined using matrix multiplication and are applied right-to-left. Inverting a transformation matrix represents a transformation out of such a space. The identity matrix $\mathbf{I}$ is a transformation that does nothing.

GLM provides functions to construct these, namely `glm::translate`, `glm::scale`, and `glm::rotate` for basic operations. The first argument to each apply their respective operations on the input matrix. GLM additionally provides utilities for constructing projection matrices, such as `glm::ortho` for orthographic projections. Multiplying transformation matrices together combines them. Transformations are applied right to left.

Transpose: The transpose of a matrix $\mathbf{A}^T$ is the matrix with its rows and columns flipped.

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^T = \begin{pmatrix} a & c \\ b & d \end{pmatrix}.$$

Furthermore,

$$(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T.$$

GLM provides `glm::transpose` to do compute the transpose.

Quaternions. Four-element number system useful for representing rotations, built as

$$\mathbf{q} = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}.$$

Specifically for rotations, quaternions always have a norm of 1, that is $\|\mathbf{q}\| = \sqrt{w^2 + x^2 + y^2 + z^2} = 1$ (also called unit quaternions).

Angle-Axis Construction: Quaternions can be constructed from an axis $\mathbf{v} = \begin{pmatrix} \mathbf{v}_x & \mathbf{v}_y & \mathbf{v}_z \end{pmatrix}^T$ and an angle θ. The construction of the quaternion $\mathbf{q}$ is given as

$$\mathbf{q} = \cos\frac{\theta}{2} + \left(\sin\frac{\theta}{2}\right)(\mathbf{v}_x\mathbf{i} + \mathbf{v}_y\mathbf{j} + \mathbf{v}_z\mathbf{k}).$$

The resulting quaternion represents a rotation around that axis. GLM provides `glm::angleAxis(angle, axis)`.

Conjugate: Similar to complex conjugate of complex numbers, for $\mathbf{q} = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$, its conjugate $\mathbf{q}^*$ is

$$\mathbf{q}^* = w - x\mathbf{i} - y\mathbf{j} - z\mathbf{k}.$$

Note since all quaternions we work with are length 1, this is equivalent to the inverse of the quaternion (i.e. $\mathbf{q}^* = \mathbf{q}^{-1}$). GLM provides the function `glm::conjugate` for this.

Inverse: The inverse of a quaternion is defined as

$$\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{\|\mathbf{q}\|^2}.$$

This has the effect of $\mathbf{q}\mathbf{q}^{-1} = 1$. Note $\mathbf{q}^{-1}\mathbf{q} \neq 1$ in general. GLM provides `glm::inverse` for this.

Multiplication: Quaternion multiplication is used to combine rotations together. Note multplication does not commute, that is $\mathbf{pq} \neq \mathbf{qp}$. You may multiply quaternions as if they were regular numbers using GLM, as `operator*` is overloaded.

Vector Transformations: Quaternions can be used to rotate vectors. If $\mathbf{x}$ is a vector treated as a quaternion (i.e. x, y, z are the vector parts and $w = 0$), then $\mathbf{x}$ rotated by $\mathbf{q}$ is

$$\mathbf{x}' = \mathbf{q}\mathbf{x}\mathbf{q}^{-1}.$$

The vector itself can be extracted from the x, y, and z components of $\mathbf{x}'$. When multiplying (via `operator*`) a vector by a quaternion in GLM, this equation is applied.

Geometry.

Area-Weighted Center: Computes the triangle area-weighted center of a mesh with triangles T, a good estimation of the mesh's center.

$$\frac{1}{\sum_{t \in T} A(t)} \sum_{t \in T} A(t) C(t),$$

where $A(t)$ is the area of a triangle and $C(t)$ is the center of the triangle. The center of a triangle is the average of its vertex positions. In the code, the triangles of a mesh are stored in `Mesh::triangles`, the vertices of a triangle are stored in `Triangle::vertices`, and a triangle's area is stored in `Triangle::area`.

Equation of Sphere: The equation for a sphere centered at the origin of radius r is given by

$$x^2 + y^2 + z^2 = r^2.$$

Normal Vector: The vector pointing away from the surface mesh, denoted $\mathbf{n}$. The normal vector (sometimes just called the normal), always has norm one (i.e. $\|\mathbf{n}\| = 1$). In code, the per-triangle normal vector is stored in `Triangle::normal` and per-vertex normal in `Vertex::normal`.

Unweighted Center: Computes the center of a mesh with vertices V.

$$\frac{1}{|V|} \sum_{\mathbf{v} \in V} \mathbf{v}.$$

This method is problematic with poorly meshed surfaces with some areas having more vertices than others. In the code, the vertices of a mesh are stored in `Mesh::vertices`.

OpenGL/Windowing. We are using the fixed-function pipeline of OpenGL, a legacy method of rendering. When consulting online resources, be careful as they may be working with modern OpenGL contexts.

Normalized Device Coordinates: Coordinate system such that $\begin{pmatrix}-1 & -1\end{pmatrix}^T$ represents the bottom-left of the viewport and $\begin{pmatrix}1 & 1\end{pmatrix}^T$ represents the top-right of the viewport. Note $\begin{pmatrix}0 & 0\end{pmatrix}^T$ is the center. Transformed to by the projection matrix $\mathbf{P}$, retrievable by `Camera::projection`. Sometimes abbreviated to NDC.

$$\mathbf{p}_{\text{ndc}} = \mathbf{P}\mathbf{p}_{\text{view}}.$$

Normal Matrix: Matrix transforming vectors. Computed as the inverse transpose of the upper 3×3 block as the model-view matrix. Formally, for the model-view matrix $\mathbf{VM}$,

$$\text{Normal Matrix} = \left((\mathbf{VM})^T_{3\times 3}\right)^{-1}.$$

When transforming vectors, they should not be translated (unlike points), so this matrix must be used. Multiplication by this matrix *does not* retain length. Therefore, if working with a unit vector (such as the surface normal), you must normalize it after applying the normal matrix to it.

Note: This matrix is unrelated to whether or not a matrix is *normal*, an different concept. The naming of this matrix comes from its use when transforming the normal vectors of a mesh.

Model-View Matrix: The product of the view matrix and the model matrix.

$$\text{Model-View Matrix} = \mathbf{VM}.$$

This matrix transforms object-space coordinates to view space. Similarly, its inverse transforms view-space coordinates to object-space.

$$\mathbf{p}_{\text{view}} = \mathbf{VM}\mathbf{p}_{\text{object}}, \quad \mathbf{p}_{\text{object}} = (\mathbf{VM})^{-1}\mathbf{p}_{\text{view}}.$$

Object Space: Coordinates local to an object. Typically, for some file these are the vertex positions stored inside the file. Transformed to by the inverse of the model matrix (from view space). Multiplying the model matrix $\mathbf{M}$ by a vertex position in object space transforms to view space. In GLM, to take the upper 3×3 block of a matrix, pass a larger matrix into the constructor for the `glm::mat3` class. In the code, the model matrix is stored in `Mesh::model`.

$$\mathbf{p}_{\text{object}} = \mathbf{M}^{-1}\mathbf{p}_{\text{world}}$$

Rendering: Rendering in OpenGL's fixed function pipeline is done through passing vertex information in between calls to `glBegin` and `glEnd`. `glBegin` takes a single argument specifying the type of primitive to be rendered (e.g. triangles, lines, quads, etc.). The function `glVertex3f` or `glVertex3fv` pass a vertex position to OpenGL to use. You may specify additional information, such as per-vertex color through `glColor3f` or `glColor3fv`.

You must set up all per-vertex information prior to calling a `glVertex` variant.

View Space: Space such that the origin is the camera's position and the vector $\begin{pmatrix}0 & 0 & -1\end{pmatrix}^T$ represents the direction the camera is facing. Transformed to by multiplying the view matrix $\mathbf{V}$ by a vector or another matrix. You can retrieve this matrix from `Camera::view`. A useful property of vectors in this space is the z component of a vector is descriptive of its direction towards or away the camera (positive values point away and negative towards).

$$\mathbf{p}_{\text{view}} = \mathbf{V}\mathbf{p}_{\text{world}}, \quad \mathbf{p}_{\text{view}} = \mathbf{P}^{-1}\mathbf{p}_{\text{ndc}}.$$

World Space: Coordinate system sitting in-between object and view space. Transformed to by the model matrix (when in object space) or the inverse of the view matrix (when in view space).

$$\mathbf{p}_{\text{world}} = \mathbf{M}\mathbf{p}_{\text{object}}, \quad \mathbf{p}_{\text{world}} = \mathbf{V}^{-1}\mathbf{p}_{\text{view}}.$$

Screen Space: Typically represents the actual 2D pixel coordinates. In most windowing systems (including GLFW which is being used), the origin is in the top-left of the window and grows left to right and top to bottom, such that the top-left is $\begin{pmatrix}0 & 0\end{pmatrix}^T$ and the bottom-right is $\begin{pmatrix}w & h\end{pmatrix}^T$, where w and h are the width and height of the window, respectively. The transformation from normalized device coordinates $\begin{pmatrix}x_{\text{ndx}} & y_{\text{ndx}}\end{pmatrix}^T$ to screen space is

$$x_{\text{screen}} = \frac{wx_{\text{ndc}} + w}{2}, \quad y_{\text{screen}} = \frac{h - hy_{\text{ndc}}}{2}.$$

Similarly, the inverse of this transformation transforms to normalized device coordinates to screen space and has the form

$$x_{\text{ndc}} = \frac{2x_{\text{screen}} - w}{w}, \quad y_{\text{ndc}} = \frac{h - 2y_{\text{screen}}}{h}.$$

GLM. The homework provides GLM, a C++ library meant for use with OpenGL and defines types and functions for performing operations with vectors and matrices. The ones that are needed the most are `glm::vec3`, `glm::vec4`, `glm::mat3`, and `glm::mat4`, representing floating point types for a 3-element vector, 4-element vector, 3×3 matrix, and 4×4 matrix, respectively. Their constructors are similar and described below.

Zero-Argument Constructors: Initializes all elements to zero.

One-Argument Constructors: For vectors, fill all elements with the argument. If a vector of greater size is passed, the extra values are discarded. For matrices, fills the diagonal.

N-Element Constructors: For vectors, fills the vector with the elements passed. Vector types are accepted and are unwrapped to fill multiple elements of the vector. If more elements are passed than necessary, the extra elements are discarded.

Operators for these are overloaded, so you may operate on them as if they are normal numeric types in C++. Note vector-vector multiplication and division is allowed here; GLM performs these operations element-wise. Note: when using GLM types, the function `glm::value_ptr` returns a pointer that is compatible

with OpenGL functions. Consult the GLM documentation (provided in the glm subdirectory) for more information on these, there are several examples present.

For quaternions, the `glm::quat` type is provided, with constructor `glm::quat(w,x,y,z)` and has a similar set of operations to vector types. Multiplying a quaternion with a `glm::vec3` is allowed and rotates the vector by the quaternion, returning another `glm::vec3`.

Some other useful GLM functions are listed:

- `glm::clamp(v, a, b)`: Clamps the vector or scalar v such that all elements are in the range $[a, b]$. If a and b are vectors, clamping is performed element-wise.
- `glm::length2(v)`: Returns the squared L2 norm of a vector (i.e. $\|\mathbf{v}\|^2$). Requires the include `<glm/gtx/norm.hpp>`.
- `glm::max(a,b)`: Returns the element-wise maximum of two vectors. That is, each element of the result $\mathbf{c}$ will have value $\mathbf{c}_i = \max(\mathbf{a}_i, \mathbf{b}_i)$.
- `glm::min(a,b)`: Returns the element-wise minimum of two vectors. That is, each element of the result $\mathbf{c}$ will have value $\mathbf{c}_i = \min(\mathbf{a}_i, \mathbf{b}_i)$.
- `glm::sign(v)`: Checks the sign of each element in the vector, returning 1 if the element if positive, -1 if it is negative, and 0 if the element is 0.

Trackball Pipeline.

(1) Project the old and new mouse positions into NDC from screen space.
(2) Project the NDC mouse coordinates onto a unit sphere ($r = 1$) at the origin (solve equation of a sphere, knowing x, y, and r).
(3) Compute the angle between the two (old and new projected mouse positions) points on the sphere.
(4) Construct a quaternion rotating from the old point to the new point, using the computed angle and the two vectors from the origin to each point. Hint: the axis of rotation must be orthogonal to the two vectors.
(5) Multiply the view angle (`Camera::_orientation`) by the quaternion to update the view.

Since quaternion multiplication does not compute, you may need to reason and experiment with the order in which you multiply as well as whether or not you want to invert your rotations.

Silhouette Pipeline.

(1) Compute the normal matrix $\mathbf{N}$.
(2) For each edge in the mesh (`Mesh::edges`):
    (a) Transform the normal vectors of the two adjacent triangles into view space using the normal matrix.
    (b) If the transformed normal vectors face different directions with respect to the camera direction, the edge is a silhouette edge.